ErrorBuddy

A beginner-friendly C compiler that explains its own errors

Lexer • Parser • Semantic Analysis • Three-Address Code • Friendly Errors

Compiler Design Project | Implemented in C++17

Problem Statement

Beginners lose hours on compiler errors they cannot read.

```
3   int a = 5;
4   int b = 10          <- real mistake
5   printf("%d", a + b);
```

```
gcc says:
program.c:5:5: error: expected
',' or ';' before 'printf'
```

Wrong line

Error points to line 5.
The fix belongs on line 4.

Jargon

"expected ... before"
Grammar terms, not plain English.

No fix

Says what is wrong, never how to fix it.

Objectives

1

Build a working compiler

Source → three-address code (TAC)

2

Cover all core phases

Lexical, Syntax, Semantic, Intermediate Code

3

Report errors like a tutor

Right line, plain words, one concrete fix

4

Use C++ only

Hand-written, standard library, no Lex / Yacc

Proposed Solution

A compiler for beginner C whose errors teach instead of confuse.

Every error is a structured object

LINE	where the fix belongs
PHASE	lexical / syntax / semantic
MESSAGE	what rule was broken, in plain English
FIX	one concrete suggestion

ErrorBuddy output, same mistake

```
Error on line 4:
Expected ';' at the end of
the statement.

Suggestion: Add a semicolon
after '10'.

(Found during: Syntax Analysis)
```

Language Scope: Beginner C

```c
#include <stdio.h>
int main() {
  int sum = 0;
  for (int i = 1; i <= 10; i++) {
    if (i % 2 == 0)
      sum = sum + i;
  }
  printf("%d\n", sum);
  return 0;
}
```

Types

int variables

Operators

+ - * / % == != < > && || !

Control flow

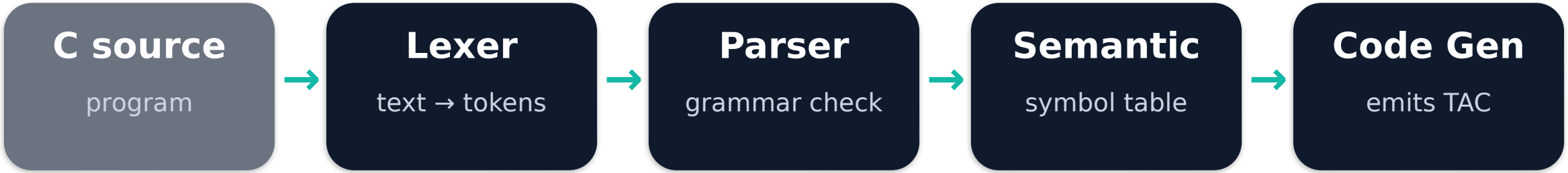
if / else, while, for, { } blocks

I/O

printf with %d, inside main()

Every valid program is real C that gcc accepts.

System Architecture



Phases 2 + 3 + 4 run together in ONE pass

Success

TAC + symbol table
+ phase summary

Error

Friendly explanation.
No partial code shown.

CompileError

One shared type used by every phase

Implementation: Phases 1 & 2

1 Lexical Analysis

Hand-written scanner, char by char
Tokens carry line numbers
Skips comments and #include
Rejects @ 2nd unclosed "string"

```
if (a == 10)
KEYWORD    if
IDENTIFIER a
OPERATOR   ==   NUMBER 10
```

2 Syntax Analysis

Recursive descent parser
One C++ function per grammar rule
Precedence built into the grammar
Left-associative: 10 - 4 - 3
else binds to nearest if

```
expression()  →  +
  term()      →  b * 2 first
   factor()   →  b , 2
```

Implementation: Phases 3 & 4

3 Semantic Analysis

Scoped symbol table: a stack of maps, one per { } block

- **Undeclared var**
- **Redeclaration**
- **Uninitialised use**
- **printf %d mismatch**

4 Three-Address Code

One operator per instruction.
Control flow = labels + jumps.

input

```
if (a > b) max = a;
else       max = b;
```

TAC output

```
  t1 = a > b
  ifFalse t1 goto L1
  max = a ; goto L2
L1: max = b
L2:
```

Implementation: Error Engine

Finds the likely cause, not just the failing token.

`if (x = 5)`	→	Did you mean '=='? '=' stores a value.
`{ ... missing }`	→	Missing '}' for the '{' opened on line 3.
`printf("%d")`	→	printf has %d but no value to print.
`retrun`	→	Did you mean the keyword 'return'?
`int b = 10`	→	Expected ';'. Reported on the line of '10'.

Token look-around | **Levenshtein edit distance** | **Line-accurate reporting**

Result: gcc vs ErrorBuddy

gcc

```
program.c:5:5: error:
expected ',' or ';'
before 'printf'
```

ErrorBuddy

```
Error on line 4:
Expected ';' at the end of
the statement.
Suggestion: Add a semicolon
after '10'.
```

✓ Right line

fix location

✓ Plain words

named rule

✓ Actionable

what to type

Tools & Testing

Tools

C++17 • standard library only
g++ • Makefile

Testing

One sample program per error type
Expected output stored for each
Automated: make test

Test coverage

- ✓ Valid program + precedence
- ✓ Undeclared variable
- ✓ Duplicate declaration
- ✓ Missing semicolon
- ✓ Two operators in a row
- ✓ Missing / extra parenthesis
- ✓ Division by zero
- ✓ Empty program

Status: 10 / 10 tests passing

Future Scope

Functions & arrays

user-defined, beyond main()

Error recovery

report several errors per run

More types

char, float

Optimisation

constant folding on TAC

Web playground

try it in the browser

Assembly backend

TAC → target code

Kept simple on purpose: int only, one function, stops at first error.

Conclusion

Complete compiler

Lexer → Parser → Semantic → TAC, built in C++

Core contribution

Errors that teach: right line, plain words, concrete fix

Impact

Less time debugging, more time learning C

ErrorBuddy: the right line, plain words, and a concrete fix. Thank you!